

Лабораторная работа №2

Базовые типы (tuple, list, set, dict), циклы (for, while)

Каждая задача должна быть помещена в отдельный файл с названием task_1.py, task_2.py в директории Lab_2_1.

Задача 1. Посчитать слова

Во входной строке записан текст (вводится пользователем). Словом считается последовательность символов (кроме пробела), идущих подряд. Слова разделены одним или большим числом пробелов или символами конца строки.

Посчитайте:

1. Сколько раз каждое слово встречается в тексте и стройте словарь {слово: количество}.
2. Количество уникальных слов.

Слова Apple и apple считаются одинаковыми.

Задача 2. Анализ списка чисел

Пользователь вводит список чисел. Числа вводятся через пробел, могут быть как целые, так и с плавающей точкой. Выведите на экран:

1. Уникальные числа.
2. Повторяющиеся числа.
3. Четные и нечетные чисел.
4. Отрицательные чисел.
5. Числа с плавающей точкой.
6. Сумму всех чисел, кратных 5.
7. Самое большое число.
8. Самое маленькое число.

Задача 3. Второе по величине число

Пользователь вводит список чисел. Найдите **второе по величине число**.

Задача 4. Сравнение списков чисел

Пользователь вводит 2 набора чисел. Выведите на экран:

1. Числа, которые присутствуют в обоих наборах одновременно.
2. Числа из первого набора, которые отсутствуют во втором, и наоборот.
3. Числа из обоих наборов, за исключением чисел, найденных в пункте 1.

Задача 5. Анаграммы

Пользователь вводит 2 слова. Напишите программу, которая проверяет, являются ли они анаграммами (первое слово может быть сформировано путем перестановки букв во втором слове).

Пример:

```
>> "listen"
```

```
>> "silent"
```

```
True
```

Задача 6. Удаление дубликатов без set()

Пользователь вводит список (любой). Удалите все дубликаты без использования set().

Задача 7. Сжатие строки

Пользователь вводит строку. Напишите программу, которая сжимает строку следующим образом: если символ X повторяется N раз, то итоговая строка должна содержать XN.

Пример:

```
>> "aaaabbbccaff"  
a4b3c2a1f2
```

Функции, декораторы, работа с файлами

Дополнительные условия и требования к решениям:

1. *Запрещается использовать встроенные функции, которые решают задачу. Например, если задача “отсортировать список”, то запрещается использовать функции sort или sorted.*
2. *Везде, где есть ввод пользователя, предполагаем, что будут вводиться корректные данные.*
3. *Форматирование строк приветствуется.*
4. *Вывод результатов и промежуточной информации на экран должен содержать пояснения, что именно выведено (если необходимо).*
5. *Каждая задача должна быть помещена в отдельный файл с названием task_1.py, task_2.py в директории Lab_2_2.*

Задача 1. Модификация списка

Напишите функцию, которая принимает список и делает его “плоским”. Используйте рекурсию. Функция должна модифицировать переданный список, а не создавать новый.

Пример:

```
>> list_a = [1, 2, 3, [4], 5, [6, [7, []], 8, [9]]]  
>> flatten_list(list_a)  
>> print(list_a)  
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Задача 2. Слияние

Напишите функцию, которая производит слияние двух словарей. Используйте рекурсию. В обоих словарях может быть любой уровень вложенности. Вложены могут быть другие коллекции: словари, списки, множества, кортежи.

Пример:

```
>> dict_a = {"a": 1, "b": {"c": 1, "f": 4}}  
>> dict_b = {"d": 1, "b": {"c": 2, "e": 3}}  
>> merge_dicts(dict_a, dict_b)  
>> print(dict_a)  
{“a”: 1, “b”: {“c”: 2, “e”: 3, “f”: 4}, “d”: 1}
```

Задача 3. Декоратор вызовов

Напишите декоратор `log_calls`, который записывает в файл время вызова, имя и аргументы вызванной функции. Один вызов функции - одна строка в файле. Декоратор должен принимать имя файла для записи в качестве параметра.

Задача 4. Транспонирование матрицы

Пользователь вводит матрицу (список списков). Напишите функцию, которая транспонирует матрицу, не изменяя входную матрицу. Транспонирование матрицы - операция над матрицей, когда ее строки становятся столбцами с теми же номерами.

Задача 5. Декоратор кэширования

Напишите декоратор `cache`, который кэширует результат вызова функции (сохраняет в словаре). Когда функция вызывается снова с теми же аргументами, декоратор должен возвращать результат из кэша, вместо того, чтобы вызывать декорированную функцию.

Задача 6. Уникальный список

Напишите функцию `unique_elements`, которая принимает вложенный список и возвращает уникальные элементы.

Пример:

```
>> list_a = [1, 2, 3, [4, 3, 1], 5, [6, [7, [10], 8, [9, 2, 3]]]]
>> unique_elements(list_a)
[1, 2, 3, 4, 5, 6, 7, 10, 8, 9]
```

Задача 7. Сортированный список

Реализуйте функцию `merge_sorted_list`, которая принимает два отсортированных списка, и возвращает новый отсортированный список, содержащий элементы из обоих списков.

Задача 8. Замер времени

Напишите декоратор `@timing`, который выводит на экран время выполнения функции в миллисекундах.

Задача 9. Проверка типов параметров

Напишите декоратор `@type_check`, который проверяет, что параметры функции соответствуют указанным типам. Если нет - выбрасывает исключение `TypeError`.

Пример:

```
@type_check(int, int)
def add(a, b):
    ...
```